

Intrinsic Facts as Routing Keys: Derive, Don't Configure — and Two Bugs From the Boundary

Abstract

1. The principle
2. Three mechanisms
3. Two bugs from the boundary
4. Comparison
5. Prescriptions

Intrinsic Facts as Routing Keys: Derive, Don't Configure — and Two Bugs From the Boundary

Research companion to the Facetwork thesis (`thesis.md`). Workshop-length. Empirical basis: Facetwork's three routing mechanisms and two production incidents from the July 2026 record.

Abstract

Distributed work-routing systems overwhelmingly route by *configuration*: operators attach labels, selectors, queue names, or topic filters to both work and workers, and correctness depends on keeping the two sides of that vocabulary synchronized by hand. We articulate a design principle used throughout Facetwork — **routing keys must be derived from intrinsic facts about the work, computed identically on both sides of the queue** — and present three mechanisms built on it: task-list isolation derived from facet namespaces, execution-environment placement derived from content-addressed dependency manifests, and capability gating derived from host classes. The principle's value is argued not from its successes but from its violations: we dissect two production incidents that occurred precisely where configuration crept back in — a configured topic filter that silently desynchronized from reality for weeks (dedicated workers claiming nothing while generalists absorbed their load), and a derived key whose consumer set could be empty (bootstraps routed to a queue nobody polls). Both fixes restored the principle: expand configuration against ground truth at the boundary, and verify a derived key has a consumer at derivation time.

1. The principle

A routing key answers: *which workers may take this work?* Every system must compute the key twice — once when work is enqueued (tagging) and once when workers poll (filtering). The design choice is where the key comes from:

- **Configured:** an operator-maintained vocabulary — Kubernetes labels/selectors, Celery queue bindings, Temporal task-queue names, Kafka topics. The two computations agree only while humans keep them agreeing.
- **Derived:** the key is a function of an *intrinsic fact* — a property the work and the worker each already possess, evaluated by the same function on both sides. Agreement is structural; there is no vocabulary to drift.

Facetwork's rule: *the same intrinsic fact on both sides of the queue*. Configuration may narrow a derived key (an override for back-compat); it must never replace one.

2. Three mechanisms

Task-list isolation from namespaces. Every facet has a qualified name; its top-level namespace is the task list. Producers tag a task for `osm.cache.Download` with list `osm` by applying `namespace_of()`; consumers poll the namespaces of the handlers they actually loaded — the same function over the same fact. A flooded `osm` backlog cannot starve `census` claims, and the isolation label *cannot* desynchronize from where the handler lives, because both are computed from the handler's name.

Environment placement from content-addressed manifests. A script facet declaring `in environment PyGeo` has its dependency set resolved at publish into exact pins and hashed. The task carries the hash; a runner advertises the hashes of environments it can actually execute (venvs present on disk, discovered — not asserted). The claim filter matches hash to hash. There is no wrong-host failure mode and no release-retry churn: a runner that cannot run the script never sees it. Note what the fact is: not a *name* (names are aspirations) but a *content address* (the frozen manifest itself) — the strongest form of intrinsic, since even two disagreeing declarations of the same name route distinctly instead of colliding.

Capability gates from host classes. Per-role server groups gate which roles *start* on which hosts (heavy imports never start on emulated minis). This is the coarse tier: the fact is the host's class, the derivation is membership, and — because routing below it is already correct — the gate affects only capacity, never correctness.

3. Two bugs from the boundary

The principle earns its keep at its violations. Both of ours are recent, production, and instructive.

Bug 1: the configured filter that lied for weeks. Runners accept a `--topics` narrowing filter — configuration, permitted as an override. The filter's globs (`census.*`) were passed *literally* into the claim query, whose matching is exact. Result: every topics-scoped runner claimed **nothing**, silently, for weeks. No errors — the generalist runners (claiming by their actual loaded-handler names, i.e. by the derived key) quietly absorbed the load, until a credential-gated task that only the dedicated runners could execute dead-lettered repeatedly and exposed the whole arrangement. Diagnosis required noticing that a runner's *claimed* work and its *configured purpose* had no overlap. The fix restored the principle at the boundary: globs are now expanded against the runner's ground truth (its loaded handler names) before entering the claim — the configuration narrows the derived vocabulary rather than replacing it. The general lesson: **a configured filter that fails closed looks exactly like a healthy idle worker**. Derived keys cannot produce this state.

Bug 2: the derived key with no consumer. Workflow bootstrap tasks route by the workflow's namespace — derived, per the principle. But a brand-new namespace implemented purely by inline scripts has no handlers anywhere, hence no runner whose derived polling set includes it: the bootstrap sat unclaimed forever on a queue nobody polls. Derivation was correct; the *consumer-existence precondition* was unchecked. The fix: at submission, verify some live worker's derived polling set covers the key; if none does, fall back to the universal queue with a printed notice. The general lesson: **derived routing needs a totality check** — every derivable key value must have a nonempty consumer set, or an explicit fallback.

4. Comparison

Label-selector systems (Kubernetes) solve a harder problem — arbitrary placement policy — and pay for it with an unverifiable vocabulary: selectors matching nothing, labels nobody selects, and no structural notion of "these two computations should have been the same function." Queue-name systems (Celery, Temporal task queues) are Bug 2 generalized: the queue name is free-floating configuration on both sides, and worker/queue mismatch is among their most common operational failures. Content-addressed dispatch appears in build systems (Nix, Bazel) for artifacts; applying it to *worker capability* (our environment hashes) is the same idea pointed at the scheduler. The trade-off is real: derived routing expresses only policies for which an intrinsic fact exists. Facetwork's answer is to keep the policy space small (isolation, capability, dependency) and refuse placement expressiveness beyond it — a luxury of purpose-built systems, but one that converts a class of operational failures into non-events.

5. Prescriptions

1. For each routing dimension, name the intrinsic fact; if none exists, you are configuring — budget for drift detection.
2. Compute the key with *one shared function* on both sides of the queue.
3. Let configuration narrow derived vocabularies, never replace them; and expand any operator syntax (globs, patterns) against ground truth before it reaches a matcher.
4. Check totality at enqueue time: a key without a live consumer is an error or an explicit fallback, not a silent queue.
5. Prefer content addresses to names when the fact has content.

Artifacts: `facetwork/runtime/task_list_routing.py`; *the environments claim path* (`claim_script_task`, `provided_environments`); *server-group gating in the fleet agent*; *incident records for the topics-glob fix and the bootstrap-list fallback with their regression suites.*